Shaheed Zulfikar Ali Bhutto Institute of Science & Technology University

DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: 2.5

Obtained Marks:

Artificial Neural Network

Task#2

Last Date of Submission: 10/14/2026

Submitted To: Hassan Mujtaba

Student Name: Qazi Safiullah

Registration Number: 24108228

Instructions: *Copied or shown assignments will be marked zero. Late submissions are not entertained in any case.*

https://github.com/safiqazi555-web/Linear-Regression-Model-Task-02.git

Logistic Regression on Two Classification Datasets

Dataset 1: NYC East River Bicycle Counts (Busy day / Quiet day). Dataset 2: Bank customers credit data (High / Low credit limit).

Each dataset is trained in Jupyter Notebook with Logistic Regression, saved as a scikit-learn Pipeline, and then used by a Gradio app and a Flask app.

Libraries Required

```
pip install pandas numpy scikit-learn matplotlib seaborn joblib gradio flask
```

Part 1: NYC East River Bicycle Counts (Jupyter Notebook)

The file has daily bike counts and no class column, so a class is created: a day is Busy when the total number of bikes is above the median, otherwise Quiet.

Step 1: Import Libraries

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns, joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, confusion_matrix, classification_report,
                             roc_auc_score, ConfusionMatrixDisplay)
```

Step 2: Load and Explore Data

```
df = pd.read_csv('nyc-east-river-bicycle-counts.csv')
print(df.shape)
df.head()
```

Output: df.shape and df.head()

```
(210, 11)
```

	Unnamed: 0	Date	Day	High Temp (°F)	Low Temp (°F)	Precipitation	Brooklyn Bridge	Manhattan Bridge	Williamsburg Bridge	Queensboro Bridge	Total
0	0	2016-04-01 00:00:00	2016-04-01 00:00:00	78.1	66.0	0.01	1704.0	3126	4115.0	2552.0	11497
1	1	2016-04-02 00:00:00	2016-04-02 00:00:00	55.0	48.9	0.15	827.0	1646	2565.0	1884.0	6922
2	2	2016-04-03 00:00:00	2016-04-03 00:00:00	39.9	34.0	0.09	526.0	1232	1695.0	1306.0	4759
3	3	2016-04-04 00:00:00	2016-04-04 00:00:00	44.1	33.1	0.47 (S)	521.0	1067	1440.0	1307.0	4335
4	4	2016-04-05 00:00:00	2016-04-05 00:00:00	42.1	26.1	0	1416.0	2617	3081.0	2357.0	9471

```
df.info()
```

Output: df.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 210 entries, 0 to 209
Data columns (total 11 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Unnamed: 0            210 non-null    int64
 1   Date                  210 non-null    str
 2   Day                   210 non-null    str
 3   High Temp (°F)        210 non-null    float64
 4   Low Temp (°F)         210 non-null    float64
 5   Precipitation         210 non-null    str
 6   Brooklyn Bridge       210 non-null    float64
 7   Manhattan Bridge      210 non-null    int64
 8   Williamsburg Bridge   210 non-null    float64
 9   Queensboro Bridge     210 non-null    float64
 10  Total                 210 non-null    int64
dtypes: float64(5), int64(3), str(3)
memory usage: 18.2 KB
```

```
df.describe()
```

Output: df.describe()

	Unnamed: 0	High Temp (°F)	Low Temp (°F)	Brooklyn Bridge	Manhattan Bridge	Williamsburg Bridge	Queensboro Bridge	Total
count	210.000000	210.000000	210.000000	210.000000	210.000000	210.000000	210.000000	210.000000
mean	104.500000	60.580000	46.413333	2269.633333	4049.533333	4862.466667	3352.866667	14534.500000
std	60.765944	11.183223	9.522796	981.237786	1704.731356	1814.039499	1099.254419	5569.173496
min	0.000000	39.900000	26.100000	504.000000	997.000000	1440.000000	1306.000000	4335.000000
25%	52.250000	55.000000	44.100000	1447.000000	2617.000000	3282.000000	2457.000000	9596.000000
50%	104.500000	62.100000	46.900000	2378.500000	4165.000000	5194.000000	3477.000000	15292.500000
75%	156.750000	68.000000	50.000000	3147.000000	5309.000000	6030.000000	4192.000000	18315.000000
max	209.000000	81.000000	66.000000	3871.000000	6951.000000	7834.000000	5032.000000	23318.000000

Step 3: Data Cleaning

```
print("Missing values:\n", df.isnull().sum())

df = df.drop(columns=['Unnamed: 0'])                # old index column
print("Duplicate rows:", df.duplicated().sum())
df = df.drop_duplicates().reset_index(drop=True)    # the same 30 days were repeated 7 times
print("Shape after removing duplicates:", df.shape)
```

Output: missing values, duplicates and shape

```
Missing values:
 Unnamed: 0             0
Date                    0
Day                     0
High Temp (°F)          0
Low Temp (°F)           0
Precipitation           0
Brooklyn Bridge         0
Manhattan Bridge        0
Williamsburg Bridge     0
Queensboro Bridge       0
Total                   0
dtype: int64
Duplicate rows: 180
Shape after removing duplicates: (30, 10)
```

```
# Clean the column names, e.g. 'High Temp (°F)' becomes 'High_Temp'
```

```
df.columns = df.columns.str.replace(r'\s*\(.*\)', '', regex=True).str.replace(' ', '_')

# Precipitation has text values: 'T' = trace of rain, '0.47 (S)' = value with a note
df['Precipitation'] = (df['Precipitation'].astype(str)
                       .str.replace(r'\s*\(S\)', '', regex=True)
                       .replace('T', '0.001')
                       .astype(float))

# Weekend feature from the date
df['Date'] = pd.to_datetime(df['Date'])
df['Weekend'] = np.where(df['Date'].dt.dayofweek >= 5, 'Yes', 'No')

df[['Date', 'High_Temp', 'Low_Temp', 'Precipitation', 'Weekend', 'Total']].head()
```

Output: cleaned columns

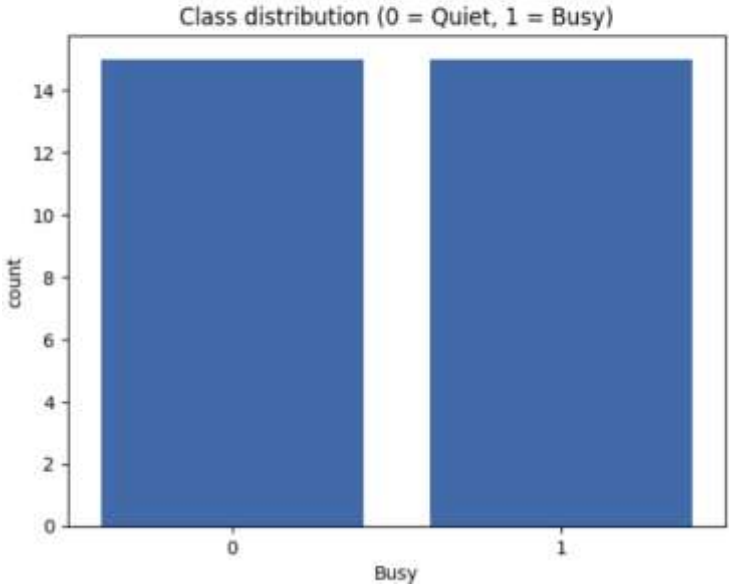
	Date	High_Temp	Low_Temp	Precipitation	Weekend	Total
0	2016-04-01	78.1	66.0	0.01	No	11497
1	2016-04-02	55.0	48.9	0.15	Yes	6922
2	2016-04-03	39.9	34.0	0.09	Yes	4759
3	2016-04-04	44.1	33.1	0.47	No	4335
4	2016-04-05	42.1	26.1	0.00	No	9471

Step 4: Create Binary Target (Busy = total bikes above the median)

```
median_total = df['Total'].median()
df['Busy'] = (df['Total'] > median_total).astype(int)   # 1 = Busy, 0 = Quiet
print("Median daily total:", median_total)
print(df['Busy'].value_counts())
sns.countplot(x='Busy', data=df)
plt.title('Class distribution (0 = Quiet, 1 = Busy)')
plt.show()
```

Output: class counts and count plot

```
Median daily total: 15292.5
Busy
0    15
1    15
Name: count, dtype: int64
```



Step 5: Select Features

The bridge counts and Total are not used as inputs because they would give the answer away.

```
num_cols = ['High_Temp', 'Low_Temp', 'Precipitation']
cat_cols = ['Weekend']

X = df[num_cols + cat_cols]
y = df['Busy']
```

Step 6: Split, Preprocess and Train

Only 30 days are available, so 30% is kept for testing.

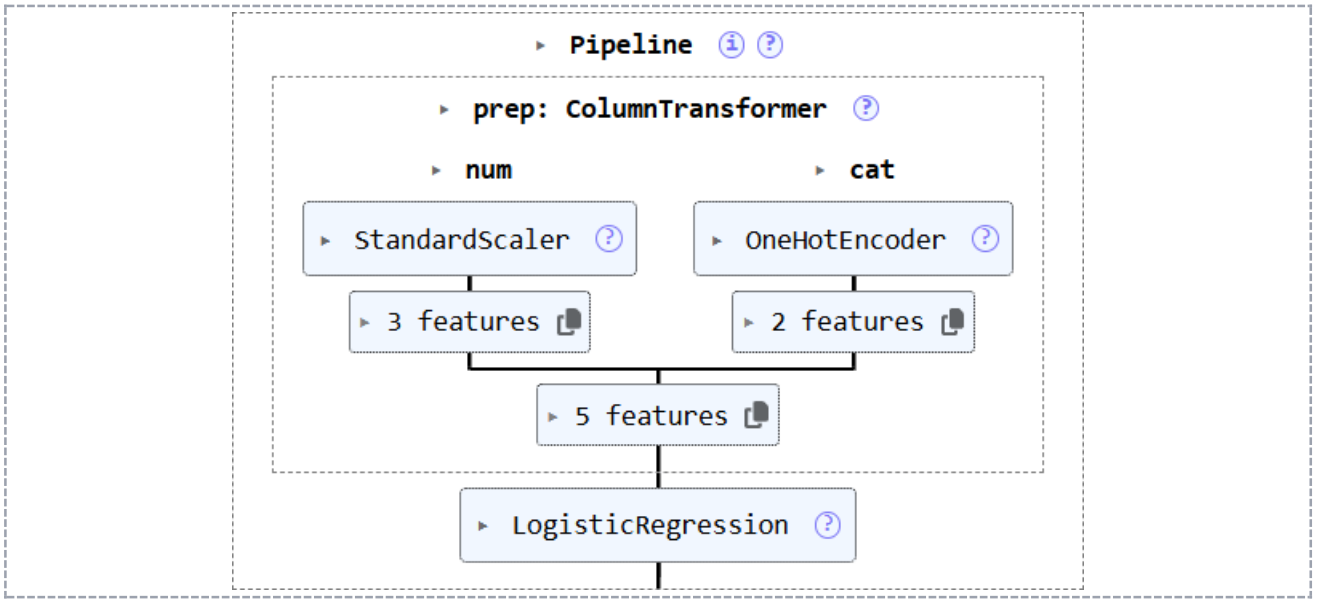
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)

preprocess = ColumnTransformer([
    ('num', StandardScaler(), num_cols),
    ('cat', OneHotEncoder(handle_unknown='ignore'), cat_cols)
])

model = Pipeline([
    ('prep', preprocess),
    ('lr', LogisticRegression(max_iter=1000))
])

model.fit(X_train, y_train)
```

Output: trained pipeline



Step 7: Model Evaluation

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Accuracy :", accuracy_score(y_test, y_pred))
print("ROC-AUC  :", roc_auc_score(y_test, y_prob))
print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=['Quiet','Busy']))
```

Output: accuracy, ROC-AUC and classification report

```
Accuracy : 0.8888888888888888
ROC-AUC  : 1.0

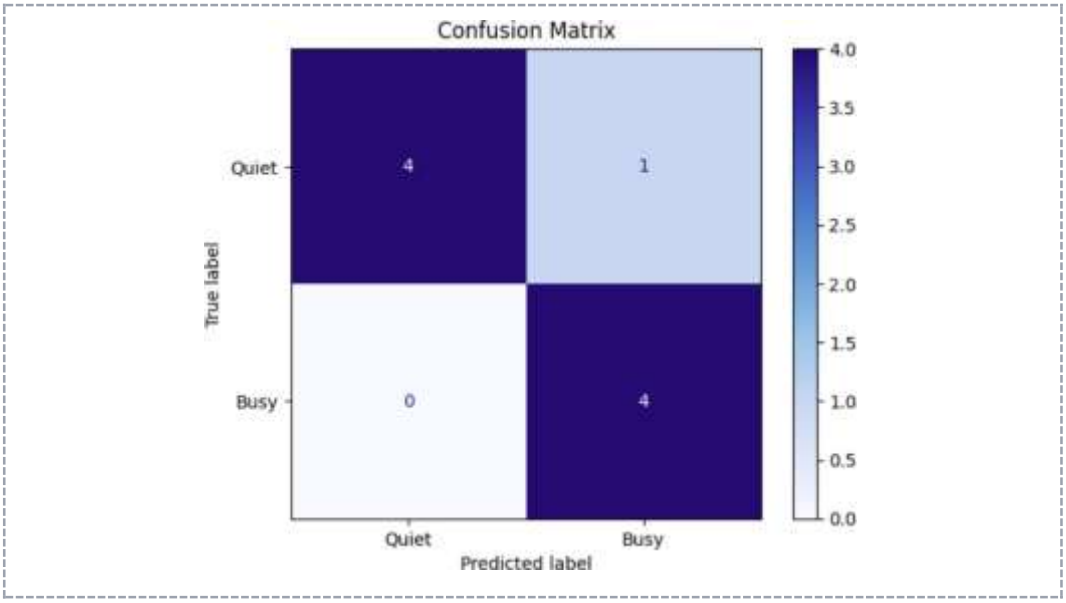
Classification Report:
              precision    recall  f1-score   support

       Quiet       1.00      0.80      0.89         5
        Busy       0.80      1.00      0.89         4

    accuracy                           0.89         9
   macro avg       0.90      0.90      0.89         9
weighted avg       0.91      0.89      0.89         9
```

```
ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred), display_labels=['Quiet','Busy']).plot(cmap='Blues')
plt.title('Confusion Matrix')
plt.show()
```

Output: confusion matrix



Step 8: Test One Prediction and Save the Model

```
sample = pd.DataFrame([{
    'High_Temp': 70, 'Low_Temp': 55, 'Precipitation': 0.0, 'Weekend': 'No'}])
print("Prediction:", "Busy" if model.predict(sample)[0] == 1 else "Quiet")
print("Probability of Busy:", model.predict_proba(sample)[0][1])

joblib.dump(model, 'bicycle_model.pkl')
print("Model saved: bicycle_model.pkl")
```

Output: sample prediction and model saved

```
Prediction: Busy
Probability of Busy: 0.8131156413887907
Model saved: bicycle_model.pkl
```

Part 2: Bank Customers Credit Limit (Jupyter Notebook)

The file has customer spending and credit columns and no class column, so a class is created: a customer has a High credit limit when credit_limit is above the median, otherwise Low. credit_limit itself is not used as an input.

Step 1: Import Libraries

```
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns, joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, confusion_matrix, classification_report,
                             roc_auc_score, ConfusionMatrixDisplay)
```

Step 2: Load and Explore Data

```
df = pd.read_csv('bank_marketing_part1_Data.csv')
print(df.shape)
df.head()
```

Output: df.shape and df.head()

```
(210, 7)
```

	spending	advance_payments	probability_of_full_payment	current_balance	credit_limit	min_payment_amt	max_spent_in_single_shopping
0	19.94	16.92	0.8752	6.675	3.763	3.252	6.550
1	15.99	14.89	0.9064	5.363	3.582	3.336	5.144
2	18.95	16.42	0.8829	6.248	3.755	3.368	6.148
3	10.83	12.96	0.8099	5.278	2.641	5.182	5.185
4	17.99	15.86	0.8992	5.890	3.694	2.068	5.837

```
df.info()
```

Output: df.info()

```
<class 'pandas.DataFrame'>
RangeIndex: 210 entries, 0 to 209
Data columns (total 7 columns):
 #   Column                        Non-Null Count  Dtype  
---  ------                        --------------  -----  
 0   spending                      210 non-null    float64
 1   advance_payments              210 non-null    float64
 2   probability_of_full_payment   210 non-null    float64
 3   current_balance               210 non-null    float64
 4   credit_limit                  210 non-null    float64
 5   min_payment_amt               210 non-null    float64
 6   max_spent_in_single_shopping  210 non-null    float64
dtypes: float64(7)
memory usage: 11.6 KB
```

```
df.describe()
```

Output: df.describe()

[6]:

	spending	advance_payments	probability_of_full_payment	current_balance	credit_limit	min_payment_amt	max_spent_in_single_shopping
count	210.000000	210.000000	210.000000	210.000000	210.000000	210.000000	210.000000
mean	14.847524	14.559286	0.870999	5.628533	3.258605	3.700201	5.408071
std	2.909699	1.305959	0.023629	0.443063	0.377714	1.503557	0.491480
min	10.590000	12.410000	0.808100	4.899000	2.630000	0.765100	4.519000
25%	12.270000	13.450000	0.856900	5.262250	2.944000	2.561500	5.045000
50%	14.355000	14.320000	0.873450	5.523500	3.237000	3.599000	5.223000
75%	17.305000	15.715000	0.887775	5.979750	3.561750	4.768750	5.877000
max	21.180000	17.250000	0.918300	6.675000	4.033000	8.456000	6.550000

Step 3: Data Cleaning

```
print(df.isnull().sum())
df = df.drop_duplicates()
df = df.dropna()
print(df.shape)
```

Output: missing values and shape

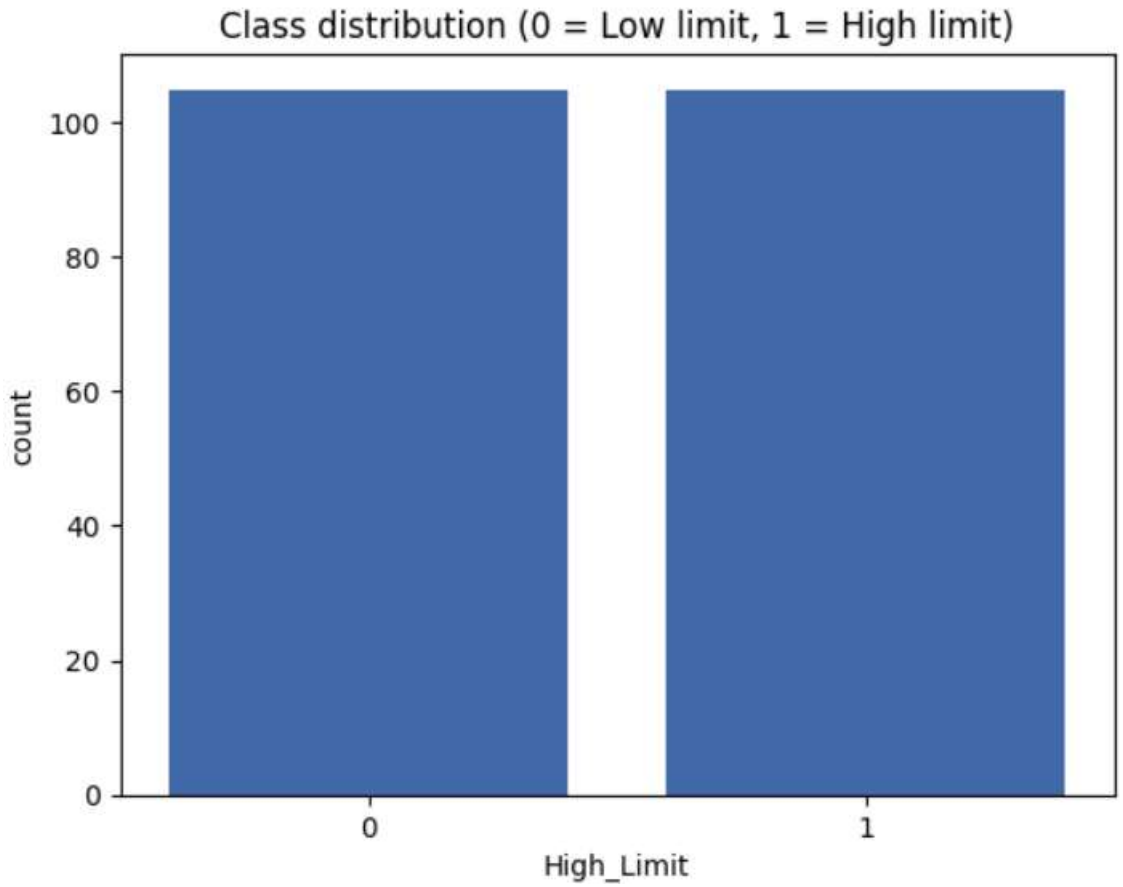
```
spending                      0
advance_payments              0
probability_of_full_payment   0
current_balance               0
credit_limit                  0
min_payment_amt               0
max_spent_in_single_shopping  0
dtype: int64
(210, 7)
```

Step 4: Create Binary Target (High limit = credit_limit above the median)

```
median_limit = df['credit_limit'].median()
df['High_Limit'] = (df['credit_limit'] > median_limit).astype(int)   # 1 = High, 0 = Low
print("Median credit limit:", median_limit)
print(df['High_Limit'].value_counts())
sns.countplot(x='High_Limit', data=df)
plt.title('Class distribution (0 = Low limit, 1 = High limit)')
plt.show()
```

Output: class counts and count plot

```
Median credit limit: 3.237
High_Limit
1    105
0    105
Name: count, dtype: int64
```



Step 5: Select Features

```
num_cols = ['spending', 'advance_payments', 'probability_of_full_payment',
            'current_balance', 'min_payment_amt', 'max_spent_in_single_shopping']


X = df[num_cols]
y = df['High_Limit']
```

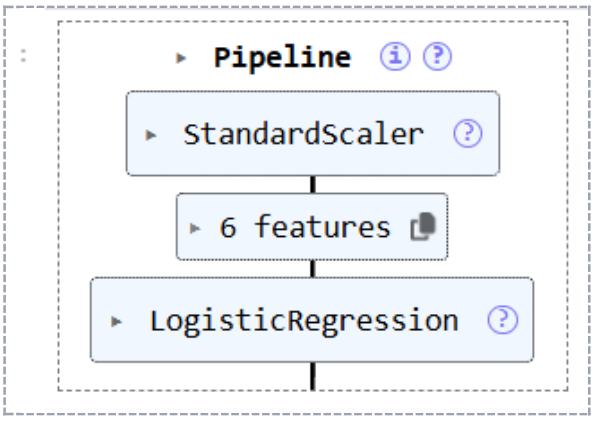
Step 6: Split, Scale and Train

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

model = Pipeline([
    ('scaler', StandardScaler()),
    ('lr', LogisticRegression(max_iter=1000))
])

model.fit(X_train, y_train)
```

Output: trained pipeline



Step 7: Model Evaluation

```
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print("Accuracy :", accuracy_score(y_test, y_pred))
print("ROC-AUC  :", roc_auc_score(y_test, y_prob))
print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=['Low limit','High limit']))
```

Output: accuracy, ROC-AUC and classification report

```
Accuracy : 1.0
ROC-AUC  : 1.0

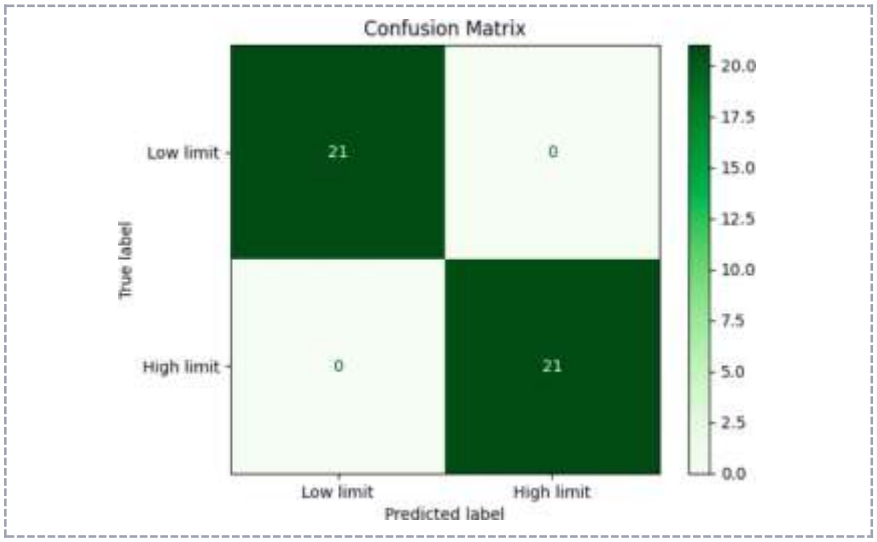
Classification Report:
              precision    recall  f1-score   support

   Low limit       1.00      1.00      1.00        21
  High limit       1.00      1.00      1.00        21

    accuracy                           1.00        42
   macro avg       1.00      1.00      1.00        42
weighted avg       1.00      1.00      1.00        42
```

```
ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred), display_labels=['Low limit','High limit']).plot(cmap='Greens')
plt.title('Confusion Matrix')
plt.show()
```

Output: confusion matrix



Step 8: Test One Prediction and Save the Model

```
sample = pd.DataFrame([{
    'spending': 15.0, 'advance_payments': 14.5, 'probability_of_full_payment': 0.88,
    'current_balance': 5.6, 'min_payment_amt': 3.0, 'max_spent_in_single_shopping': 5.2}])
print("Prediction:", "High limit" if model.predict(sample)[0] == 1 else "Low limit")
print("Probability of High limit:", model.predict_proba(sample)[0][1])

joblib.dump(model, 'bank_model.pkl')
print("Model saved: bank_model.pkl")
```

Output: sample prediction and model saved

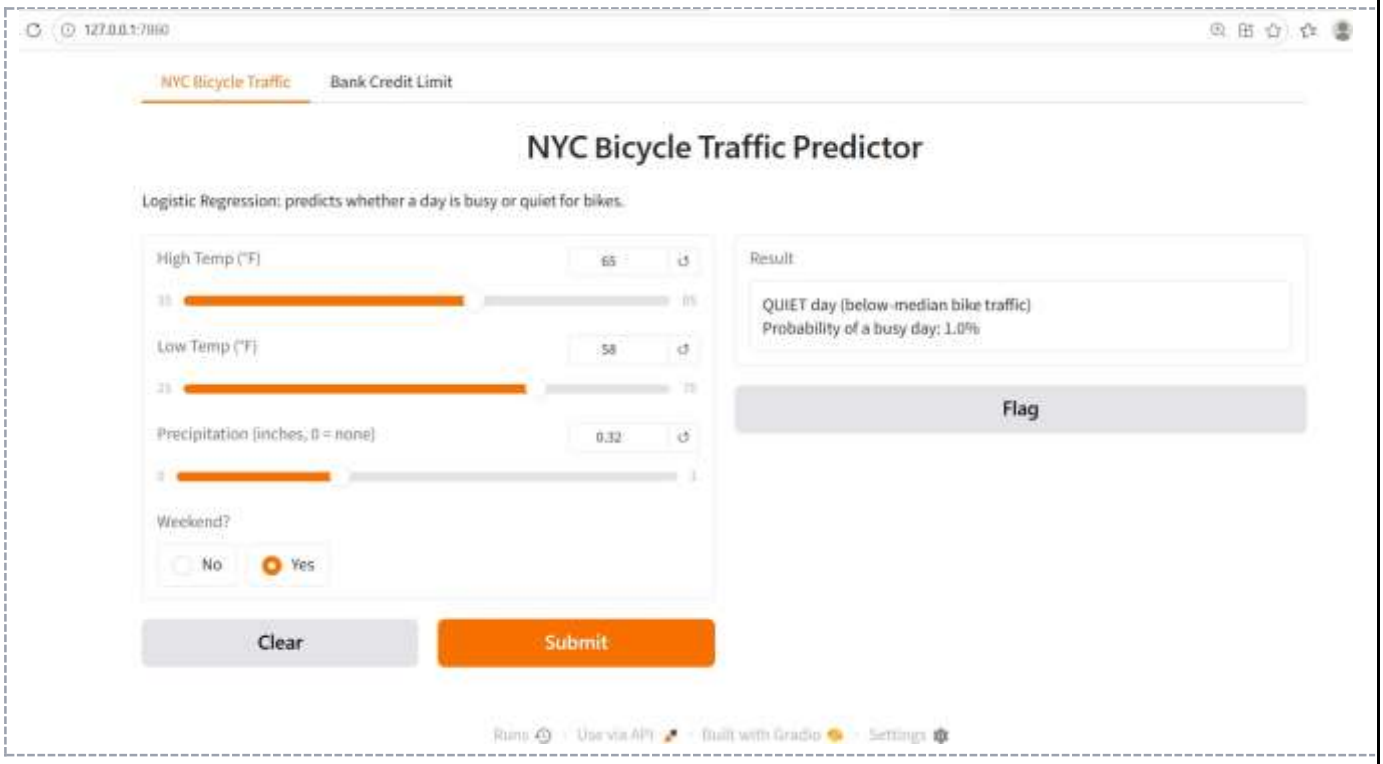
```
Prediction: High limit
Probability of High limit: 0.7089647766713099
Model saved: bank_model.pkl
```

Part 3: Gradio User Interface

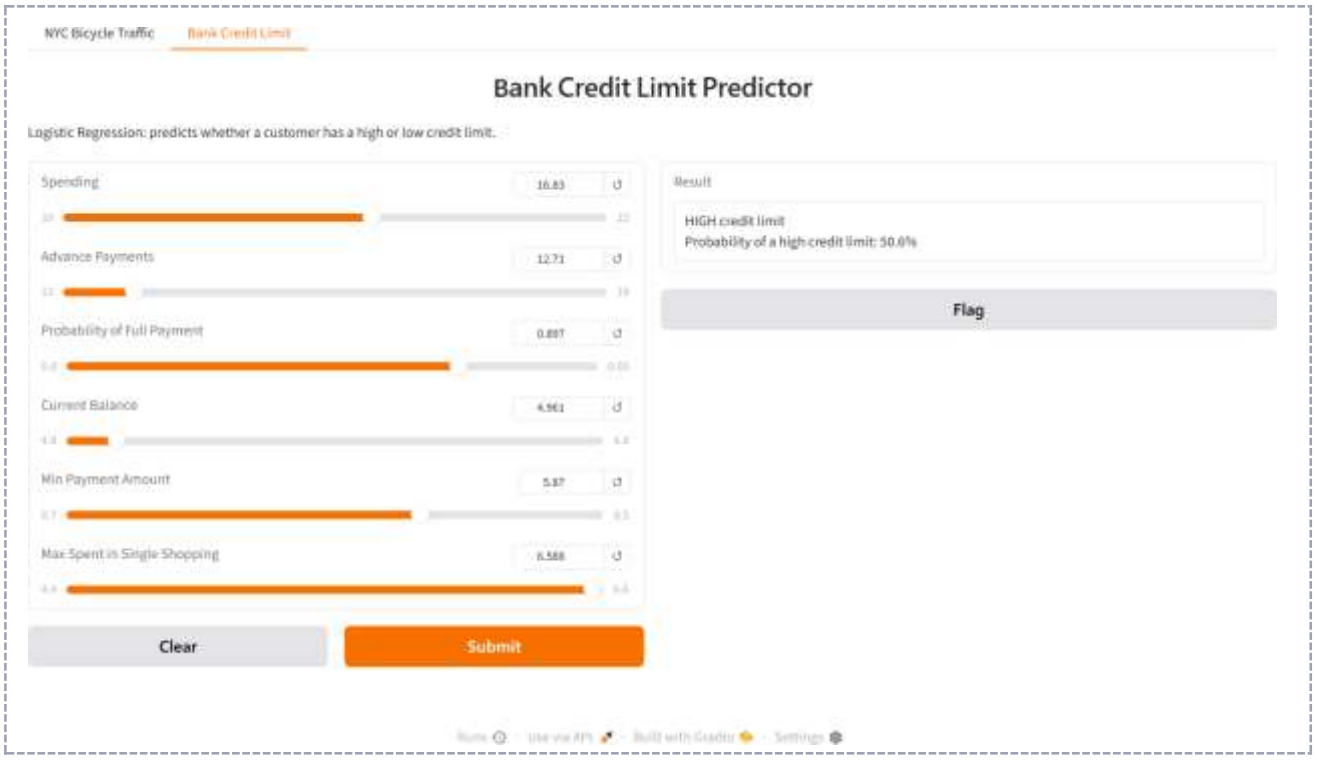
File: gradio_app.py. It loads both saved models and shows one tab per dataset. Run with: python gradio_app.py

Gradio Output

Output: Gradio, NYC Bicycle Traffic tab with a prediction



Output: Gradio, Bank Credit Limit tab with a prediction



Part 4: Flask User Interface

Folder structure: flask_app/ contains app.py and a templates/ folder with index.html and form.html. Both .pkl model files are in the project folder above it. Run with: python app.py (then open http://127.0.0.1:5000)

app.py

templates/index.html

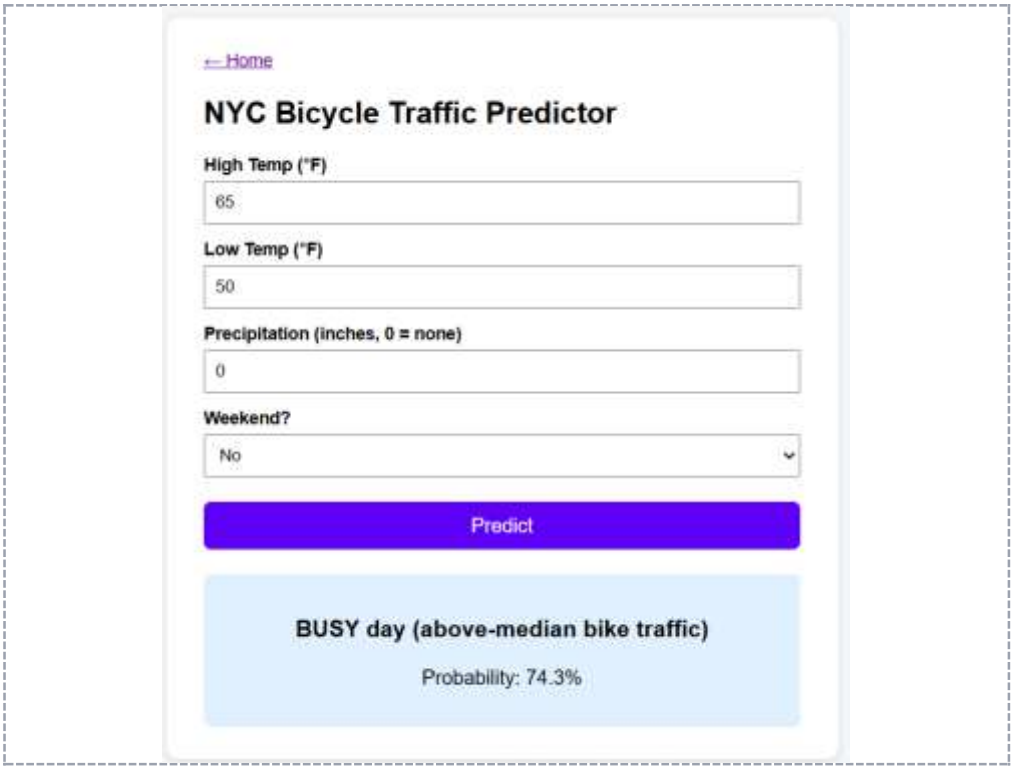
templates/form.html

Flask Output

Output: Flask, home page



Output: Flask, NYC Bicycle Traffic form with a prediction



Output: Flask, Bank Credit Limit form with a prediction

